Solveur Adaptatif pour le Problème de Coloration de Graphes Branch & Bound Basé sur la Densité

SID02 — Équipe 02

Merabtine Aya Malek Kebbal Malek Koroghli Fadhila Diaf Lina Malek Yasmine

Résumé : Ce rapport présente la conception d'un solveur adaptatif pour le Problème de Coloration de Graphes (GCP). Chaque instance est classifiée automatiquement selon sa densité, puis résolue par la variante Branch & Bound la plus adaptée parmi trois algorithmes DSATUR spécialisés. Pour chacun, nous détaillons la fonction d'évaluation, la borne supérieure initiale, la règle de choix du nœud et la stratégie d'exploration de l'arbre. Une comparaison avec la Programmation Dynamique de Lawler [4] complète l'étude.

1. Stratégie Adaptative par Densité

La densité d'un graphe $G = (V, E)$, définie par $d(G) = \dfrac{2|E|}{|V|(|V|-1)}$, détermine automatiquement l'algorithme appliqué selon les seuils suivants.

Classe	Densité	Algorithme	Référence
Faible	$d < 0.3$	DSATUR VB1	[1]
Moyenne	$0.3 \leq d < 0.7$	DSATUR VB2+O2	[2]
Forte	$d \geq 0.7$	DSATUR-χ_{GA}	[3]

Justification. En densité faible, peu de contraintes rendent l'arbre B&B naturellement petit et VB1 suffit. En densité moyenne, l'ordre O2 et la règle VB2 atteignent rapidement les feuilles de l'arbre. En densité forte, le complémentaire $\bar{G}$ est creux, rendant le graphe auxiliaire G_A petit et la borne χ_{GA} à la fois serrée et rapide à calculer.

2. Algorithme 1 — DSATUR VB1 (Brélaz, 1979 [1], densité faible)

2.1 Borne inférieure — Fonction d'évaluation

La borne inférieure est obtenue par une heuristique gloutonne de clique en $O(n^2)$: à partir du sommet de plus grand degré, on ajoute itérativement le voisin maximisant le nombre de voisins communs avec la clique courante. La taille ω obtenue satisfait $\omega(G) \leq \chi(G)$.

2.2 Borne supérieure initiale

L'heuristique DSATUR fournit la borne supérieure : $\mathrm{DSAT}(v) = |\{c(u) \mid u \in \mathcal{N}(v),\ u \text{ coloré}\}|$, couleur la plus petite disponible, donnant $\chi_h \geq \chi(G)$. Une réduction de graphe est ensuite appliquée : tout sommet v tel que $\deg(v) < \omega - 1$ est supprimé itérativement sans modifier χ.

2.3 Choix du nœud — Règle VB1

On sélectionne $v^* = \arg\max_{v \notin C} \mathrm{DSAT}(v)$, tie-break par **degré total** $|\mathcal{N}(v)|$. Le degré total reflète l'importance structurelle globale ; traiter tôt les sommets très connectés réduit les conflits futurs.

2.4 Stratégie d'exploration

Les couleurs sont essayées en ordre **croissant** $1, 2, \ldots, \tilde{k}, \tilde{k}+1$, favorisant la réutilisation des couleurs existantes. L'élagage intervient dès que $\tilde{k} \geq \chi$; le B&B s'arrête si $\chi = \omega$.

3. Algorithme 2 — DSATUR VB2+O2 (Méndez-Díaz & Zabala, 2006 [2], densité moyenne)

3.1 Borne inférieure — Fonction d'évaluation

Clique gloutonne identique à VB1 : $\omega(G) \leq \chi(G)$, $O(n^2)$.

3.2 Borne supérieure initiale

DSATUR **multi-départ** : plusieurs sommets de départ sont testés et la meilleure coloration est retenue. Cette approche fournit une borne plus serrée qu'en VB1 et déclenche des élagages plus précoces.

3.3 Choix du nœud — Règle VB2

On sélectionne $v^* = \arg\max_{v \notin C} \mathrm{DSAT}(v)$, tie-break par **degré résiduel** $|\mathcal{N}(v) \setminus C|$, où $\mathcal{N}(v)$ désigne l'ensemble des voisins du sommet v (sommets adjacents). Ce critère reflète les contraintes futures immédiates plutôt que la structure globale.

3.4 Stratégie d'exploration — Ordre O2

La nouvelle couleur est testée **en premier** : $\tilde{k}+1,\ 1,\ 2,\ldots,\tilde{k}$. Cela permet d'atteindre rapidement une feuille de l'arbre et de mettre à jour χ tôt. L'élagage est renforcé par une borne résiduelle ω_R calculée sur le sous-graphe non coloré : le nœud est élagué si $\tilde{k}+\omega_R \geq \chi$, condition strictement plus forte que l'élagage standard.

4. Algorithme 3 — DSATUR-χ_{GA} (Furini et al., 2016 [3], densité forte)

4.1 Borne inférieure — Fonction d'évaluation

La clique est calculée par **Bron-Kerbosch** (avec timeout). Les sommets de la clique sont pré-colorés $1\ldots\omega$ et définissent l'ordre prioritaire de traitement. Si $\omega=\chi_h$ à la racine, le B&B s'arrête immédiatement.

4.2 Borne supérieure initiale

DSATUR heuristique avec ordre prioritaire : sommets de la clique en premier, puis autres sommets par degré décroissant.

4.3 Borne forte χ_{GA}

Par le théorème de Cornaz & Jost (2008) :

$$\alpha(G_A)+\chi(G^C)=|V^C| \quad\Longrightarrow\quad \chi_{GA}(G^C)=|V^C|-\alpha(G_A)$$

G_A est construit depuis $\bar{G}$: chaque arc orienté devient un sommet de G_A. Le nombre de stabilité $\alpha(G_A)$ est calculé par résolution d'un programme linéaire en nombres entiers (MIP) :

$$\max\sum_v x_v \quad \text{s.c.} \quad x_u+x_v\leq 1\ \ \forall\,(u,v)\in E(G_A),\ \ x_v\in\{0,1\}$$

Sur les graphes denses : $\bar{G}$ creux $\Rightarrow$ G_A petit $\Rightarrow$ MIP rapide $\Rightarrow$ borne serrée $\Rightarrow$ élagages massifs. L'égalité de Cornaz-Jost étant exacte, la borne n'est pas une approximation.

4.4 Choix du nœud et stratégie d'exploration

On sélectionne le sommet de DSAT maximal, tie-break par l'ordre initial (clique en priorité). Les couleurs existantes faisables sont testées d'abord, puis la nouvelle couleur. Le calcul de χ_{GA} est activé uniquement si la stratégie $\varphi_{u,g}$ est satisfaite : $u^*\geq u$ *et* $\tilde{g}=\chi-\tilde{k}\geq g$, avec les paramètres optimaux $u=2$, $g=1$ (Furini et al., 2016). Des garde-fous mémoire sont intégrés (relaxation LP si $G_A>1$ Go).

5. Résultats Expérimentaux — Instances DIMACS

Instance	n	d	**Classe**	**Algo recommandé**	$\chi(G)$
`DSJC125.1.col`	125	0.10	Faible	DSATUR VB1	5
`myciel3.col`	11	0.36	Moyenne	VB2 + O2	4
`school1.col`	385	0.13	Faible	DSATUR VB1	14
`DSJC125.9.col`	125	0.90	Forte	DSATUR-χ_{GA}	44

L'algorithme recommandé par le classificateur est le plus performant sur chaque classe. Sur `DSJC125.9` (forte densité), l'Algo. 3 explore seulement 977 nœuds en $\approx$ 307s, tandis que les Algos. 1 et 2 atteignent le timeout de 500s sans certifier l'optimalité, illustrant la supériorité de χ_{GA} sur les graphes denses.

6. Synthèse Comparative des Trois Algorithmes B&B

Composante	**VB1** ($d<0.3$)	**VB2+O2** (0.3–0.7)	χ_{GA} ($d\geq 0.7$)
Borne inf.	Clique gloutonne $O(n^2)$	Clique gloutonne $O(n^2)$	Bron-Kerbosch + timeout
Borne sup.	DSATUR 1 départ	DSATUR multi-départ	DSATUR heuristique
Choix nœud	DSAT + degré total	DSAT + degré résiduel	DSAT + ordre initial
Ordre couleurs	Croissant $1\ldots\tilde{k}+1$	O2 : $\tilde{k}+1,\ 1\ldots\tilde{k}$	Existantes puis nouvelle
Élagage	$\tilde{k}\geq$ borne_sup	Standard + ω_R	Standard + χ_{GA} (MIP)
Réduction	deg $<\omega-1$	deg $<\omega-1$	Pré-coloration clique

Le B&B adaptatif est nettement supérieur en vitesse sur les grandes instances (100×–1000× jusqu'à $\sim$ 200 sommets). L'approche adaptative par densité maximise l'efficacité du B&B sur chaque classe d'instances.

7. Programmation Dynamique — Algorithme de Lawler (1976) (De Lima & Carmo, 2018 [4])

7.1 Principe et Récurrence

L'algorithme de Lawler (1976) résout le GCP de manière exacte en exploitant la propriété que tout coloriage optimal se décompose en un stable I (une couleur) plus le coloriage optimal du sous-graphe restant $G[S \setminus I]$. La mémorisation sur les 2^n masques binaires évite les recalculs :

$$\chi(G[S]) = 1 + \min_{I \in \mathcal{IS}(G[S])} \chi(G[S \setminus I]), \qquad \chi(\emptyset) = 0$$

Complexité : $O(2.4423^n)$ en temps, $O(2^n)$ en espace, ce qui limite son usage pratique à $n \leq 25$ sommets.

7.2 Comparaison B&B vs Programmation Dynamique de Lawler

La comparaison avec le B&B adaptatif met en évidence une **complémentarité fondamentale** entre les deux paradigmes, mesurée sur trois critères objectifs.

Critère	**B&B adaptatif**	**DP Lawler**
Prévisibilité (CV%)	CV 139–316% pour $n \geq 14$, instable selon la topologie	CV 23–35%, stable quel que soit le graphe
Pire cas ($t_{\max}/t_{\text{moy}}$)	Ratio 4.5–6.8×, sans garantie	Ratio 1.25–1.65×, déterministe
Dépendance structurelle	Temps croissant avec le gap $\chi_{\text{heur}} - \omega$	Temps $\sim f(n)$ uniquement, indépendant
Vitesse moyenne	100–1000× plus rapide ($n \lesssim 200$)	Lent mais borné par $O(2.4423^n)$
Limite de taille	Jusqu'à ~ 200 sommets	$n \leq 25$ (mémoire 2^n)

7.3 Résultats Expérimentaux

Instance	n_{test}	**CV% DP**	**CV% B&B**	**Observation**
`myciel3`	11	32.8%	282.8%	DP plus prévisible
`DSJC125.1`	18	51.5%	0.0%	B&B trivial sur sous-graphes
`School1`	18	39.7%	0.0%	B&B trivial sur sous-graphes

Sur `myciel3` ($n = 11$), la DP est nettement plus prévisible (CV 32.8% contre 282.8%). Sur les grandes instances, la DP ne peut traiter le graphe complet (2^n dépasse la mémoire) ; sur les sous-graphes induits de taille 18, le B&B résout chaque tirage quasi instantanément (CV = 0.0%), ces instances étant triviales à cette échelle.

Conclusion : B&B et DP sont complémentaires : le B&B pour les grandes instances ($n \lesssim 200$), la DP pour ses garanties formelles — variance faible, pire cas borné en $O(2.4423^n)$ et indépendance structurelle totale vis-à-vis de la topologie du graphe.

References

[1] Brelaz, D. (1979). New methods to color the vertices of a graph. *Communications of the ACM*, 22(4), 251–256. `https://doi.org/10.1145/359094.359101`

[2] Mendez-Diaz, I. & Zabala, P. (2006). A Branch-and-Cut algorithm for graph coloring. *Discrete Applied Mathematics*, 154(5), 826–847. `https://doi.org/10.1016/j.dam.2005.05.021`

[3] Furini, F., Gabrel, V. & Ternier, I.-C. (2016). An Improved DSATUR-Based Branch-and-Bound Algorithm for the Vertex Coloring Problem. *INFORMS Journal on Computing*, 29(4), 686–701. https://www.researchgate.net/publication/..

[4] De Lima, A.M. & Carmo, R. (2018). Exact Algorithms for the Graph Coloring Problem. *Revista de Informática Teórica e Aplicada (RITA)*, 25(4), 57–73. `https://doi.org/10.22456/2175-2745.80721`